Semantic segmentation of geospatial imagery with MMSegmentation

A course for students and colleagues: environment, data, pipelines, models, training, evaluation and GIS-ready prediction
Running example: *Acacia tortilis* crown mapping from UAV imagery; further examples for buildings, roads and multispectral land cover

Version 1.0 · 09 September 2026
Companion repository: github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping (folder tutorial/)
Software: MMSegmentation 1.2.2 · MMEngine 0.10.7 · MMCV 2.1.0 · PyTorch 2.6.0 (CUDA 11.8)

Prepared by Mohamed Barakat A. Gibril
GIS and Remote Sensing Center, Research Institute of Sciences and Engineering, University of Sharjah
Contact: mbgibril@sharjah.ac.ae

Project datasets referenced in this document are not publicly shareable.

Contents

0 How to use this tutorial

0.1 Who it is for

Three readers, three paths.

Reader	Goal	Path
A student starting a segmentation project	train a first model on own tiles within a week	Chapters 2, 5, 6, 8, 9 in order, then the exercises of Appendix D
A research assistant taking over a project	reproduce, retrain, compare and deploy models	Chapters 3, 4, 7, 8, 9, 10, 12
A colleague evaluating MMSegmentation for a new task	understand what changes between tasks and sensors	Chapters 1, 4, 6, 7, 11

Prerequisites: Python at the level of writing functions and reading tracebacks; familiarity with raster GIS concepts (CRS, pixel size, bands, NoData); a basic idea of what a convolutional network is. Nothing about MMSegmentation is assumed.

0.2 Learning outcomes

After working through the tutorial the reader can:

1. explain what each folder of MMSegmentation contains and how a training run flows through the library;
2. prepare image–mask tiles from GIS layers and verify them;
3. write a dataset configuration for 8-bit RGB and for 16-bit multispectral data, including augmentation pipelines;
4. select, adapt and train at least three different architectures on the same data and compare them with consistent metrics;
5. diagnose common failures (data, memory, convergence) from logs;
6. produce georeferenced predictions and vector outputs for GIS.

0.3 Conventions

- Commands are shown for the team's container (chapter 2): the repository is at `/workspace/U-MV`, data at `/data`, checkpoints at `/weights`. On a bare Linux/conda installation replace these with local paths.
- Configuration snippets are Python (MMSegmentation configs are Python files). `...` marks omitted lines.
- **Checkpoint** boxes state what should be observed before continuing. **Pitfall** boxes record verified failure modes. **Exercise** boxes propose short tasks; solutions are in Appendix D.
- Terms in *italics* at first use are defined in the glossary (Appendix C).

0.4 Versions

MMSegmentation 1.2.2, MMEngine 0.10.7, MMCV 2.1.0, PyTorch 2.6.0 (CUDA 11.8). Every signature and file path in this tutorial was checked against these versions; the OpenMMLab 0.x API (`mmseg 0.30` and earlier) is different and its tutorials do not apply.

1 Semantic segmentation and the OpenMMLab stack

1.1 What semantic segmentation does

Semantic segmentation assigns a class label to every pixel of an image. For mapping it is the operation that turns an orthomosaic into a thematic raster: tree crown / not crown, building / road / water / bare soil, and so on. The output has the same size as the input and, when the input is georeferenced, the prediction inherits its coordinate reference system and can be vectorised directly into GIS layers.

Contrast with two neighbouring tasks. *Object detection* returns bounding boxes (useful for counting trees, poor for area). *Instance segmentation* returns a separate mask per object (Mask R-CNN, Mask2Former in instance mode); it is needed when touching objects must be separated, at a higher annotation and computational cost. Semantic segmentation is the right choice when the map is the product: area, extent, cover fraction, or a raster layer for further GIS analysis.

1.2 How a segmentation network is organised

Almost every modern architecture is an *encoder–decoder*:

- The **encoder** (backbone) reduces spatial resolution stage by stage (strides 4, 8, 16, 32) while increasing the number of feature channels. It is usually pretrained on ImageNet, which is why 5 000 tiles can be enough to train a good model. Encoders differ in their inductive bias: convolutional networks (ResNet, ConvNeXt, U-Net) favour local texture; transformers (Swin, MiT, ViT) model long-range context; state-space models (VMamba, MambaVision) offer long-range modelling at lower cost.
- The **decoder** (decode head) fuses the multi-scale features back to a pixel-wise prediction: ASPP (DeepLabV3+), pyramid pooling (PSPNet), feature pyramid (UPerNet), lightweight MLP (SegFormer), U-Net skip connections.
- The **loss** compares the prediction with the reference mask: cross-entropy (per-pixel classification), Dice (region overlap, robust to imbalance), Lovasz (a surrogate of IoU), Focal (down-weights easy pixels).

In the team's own work (Gibril et al., 2026) the encoder is MambaVision and the decoder a light U-Net (Figure 1); the same encoder was combined with four decoders to show that decoder complexity matters less than the encoder for crown mapping.

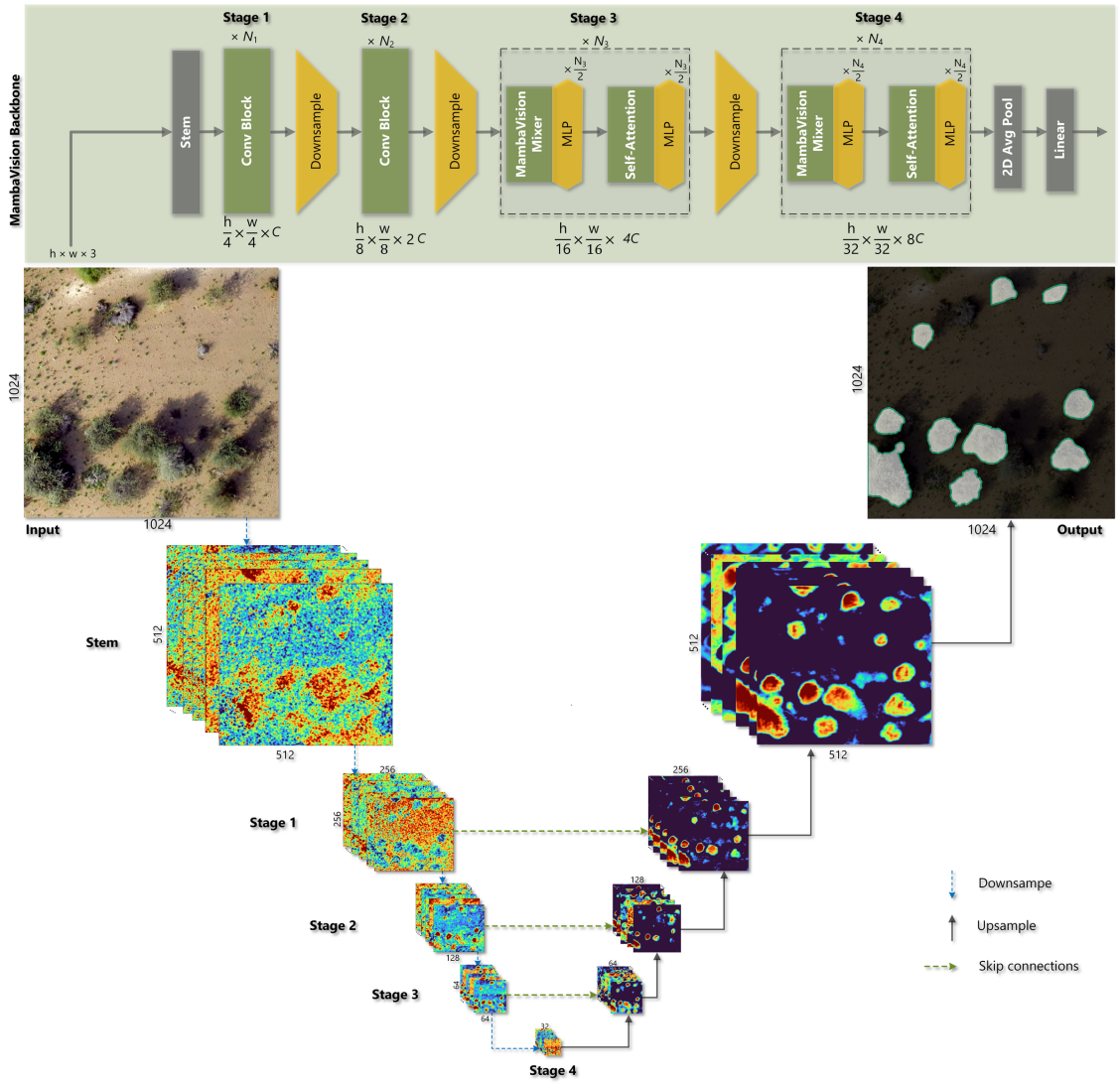

Figure 1. Encoder–decoder structure illustrated by the team's U-MV network: a four-stage MambaVision encoder (strides 4 to 32) and a U-Net decoder with skip connections applied to a 1024 × 1024 UAV tile.

1.3 Why a framework, and why MMSegmentation

Writing training loops by hand is instructive once and error-prone thereafter. A framework fixes the pieces that are the same for every project: data loading with augmentation, distributed and mixed-precision training, checkpointing, logging, evaluation, and a model zoo with pretrained weights. MMSegmentation is the segmentation library of the OpenMMLab ecosystem:

Package	Role	Version used
MMEngine	training engine: `Runner`, hooks, config system, registries, checkpoints, logging	0.10.7
MMCV	computer-vision operators (compiled CUDA ops), image transforms, `ConvModule` etc.	2.1.0
MMSegmentation	datasets, pipelines, backbones, decode heads, losses, metrics for semantic segmentation	1.2.2
MMDetection	detection and instance segmentation; provides the Mask2Former pixel decoder used by MMSegmentation's Mask2Former	3.3.0 (optional)
MMPretrain	classification backbones (ConvNeXt, ...) importable into MMSegmentation	1.2.0 (optional)

Its strengths for geospatial work: 40+ architectures with one training script; declarative configuration files that document an experiment completely; sliding-window inference for large images; metrics per class; and an easy route to add custom components (the team's MambaVision backbone is registered in ~100 lines). Its weaknesses: the 8-bit RGB assumption in several transforms (addressed in chapter 6), and a learning curve caused by the config system (chapter 4 exists to flatten it).

1.4 Vocabulary

Term	Meaning
tile / patch	a fixed-size crop (e.g. 512 × 512 or 1024 × 1024 px) of an orthomosaic used as one training sample
mask / annotation / ground truth	single-band raster with the class index of every pixel
ignore index	label value (255) excluded from loss and metrics: unlabeled or padded pixels
iteration	one optimiser step on one batch; MMSegmentation schedules are iteration-based
epoch	one pass over the training set (= tiles / batch size iterations)
mIoU	mean over classes of intersection-over-union; the standard segmentation metric
checkpoint	saved model weights (and optimiser state) at a given iteration
config	the Python file that defines data, model, schedule and runtime of one experiment
registry	a name → class table; `type='PSPHead'` in a config is looked up in the model registry
work directory	the folder where a run writes logs, checkpoints and the resolved config

1.5 Exercise 1

The *Acacia* paper trained on 1024 × 1024 tiles at 2.5 cm GSD with batch size 2 for 100 000 iterations. How many epochs is that for 26 615 tiles? How many for the 4 893 tiles of the archived training split? What does this imply for the number of iterations you should plan when you reuse the schedule on the smaller set? (Answer in Appendix D.)

2 The team environment

2.1 The standard setup

All segmentation work in the team runs inside one Docker image on Windows 11 workstations with WSL2 (Ubuntu) and an NVIDIA GPU (TITAN RTX or RTX A5000, 24 GB). The image is defined in `docker/Dockerfile` of the U-MV repository and contains the complete stack:

Layer	Content
base	`pytorch/pytorch:2.6.0-cuda11.8-cudnn9-devel` (PyTorch, CUDA 11.8, nvcc)
OpenMMLab	MMEngine 0.10.7, MMCV 2.1.0 (compiled), MMSegmentation 1.2.2
encoders	mamba-ssm 2.2.4 (MambaVision kernels), timm, transformers
geospatial	GDAL 3.10, rasterio, geopandas, shapely, pyproj (conda-forge)
team code	the `umv` package (custom backbone, decoder, loaders, tools) installed editable

Working inside a container has one purpose: everyone runs the same versions, so a config that trains on one workstation trains identically on another, and results can be compared across people and years.

2.2 One-time installation

1 Windows: install the NVIDIA driver (≥ 520), Docker Desktop with the WSL2 backend, and enable the Ubuntu distribution in *Settings → Resources → WSL integration*. Set `memory=48GB` in `%USERPROFILE%\.wslconfig` and run `wsl --shutdown` once.

2 Ubuntu (WSL2) terminal:

```
cd ~
git clone https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping.git
cd U-MV-Acacia-tortilis-Crown-Mapping
cp docker/.env.example docker/.env      # edit DATA_DIR and WEIGHTS_DIR (next section)
docker compose --env-file docker/.env -f docker/docker-compose.yml build   # 30-60 min once
```

3 Verify:

```
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
python tools/verify_install.py            # RESULT: OK
python -m pytest tests -q                 # all passed
```

Checkpoint. `verify_install.py` prints `CUDA available: True` with the GPU name, `mamba_ssm ... ok` and `mmcv.ops ... ok`.

A conda installation without Docker is documented in `docs/01_installation.md` and is acceptable for laptops without a GPU (inference and small tests only).

2.3 Where data and projects live

The container sees three locations:

Inside the container	Host (set in `docker/.env`)	Purpose
`/workspace/U-MV`	the repository clone	code, configs, `work_dirs/` (training outputs)
`/data`	`DATA_DIR`	datasets: one sub-folder per project, e.g. `/data/acacia`, `/data/buildings`
`/weights` (read-only)	`WEIGHTS_DIR`	shared checkpoints and archived work directories

Recommended host layout on the local SSD (never train from a network share):

```
D:\DL_Data\                          <- DATA_DIR = /mnt/d/DL_Data
├── acacia\{img_dir, ann_dir}\{train, val, test2, Generalizability}
├── buildings\{img_dir, ann_dir}\{train, val, test}
└── lulc_s2\{img_dir, ann_dir}\{train, val, test}
D:\DL_Weights\                       <- WEIGHTS_DIR = /mnt/d/DL_Weights
└── acacia\mambavision-s_generic-unet_acacia-88\...
```

Project configs live in the repository under `projects/<name>/` (created from `tutorial/templates/project`), and outputs under `work_dirs/<name>/<model>/`. Commit `projects/*/configs`; never commit `work_dirs/` or data.

2.4 Daily use

```
cd ~/U-MV-Acacia-tortilis-Crown-Mapping
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
# ... work ...
exit
```

Long trainings should run inside `tmux` so that closing the terminal does not stop them: `tmux new -s train`, start the job, detach with Ctrl+B then D, re-attach with `tmux attach -t train`. A second shell into a running container: `docker exec -it <container id> bash` (`docker ps` lists ids).

Useful environment variables (set in `docker/.env` or before a command):

Variable	Effect
`HF_HUB_OFFLINE=1`	never contact the Hugging Face Hub (after `tools/download_backbones.py`)
`CUDA_VISIBLE_DEVICES=0`	select the GPU on multi-GPU hosts
`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`	reduces fragmentation-related OOM (set by compose)

2.5 Hardware budget

Configuration	Typical VRAM	Notes
DeepLabV3+ R50, 512², batch 4	~10 GB	fp32
SegFormer-B2, 512², batch 4	~8 GB	
UPerNet R50, 1024², batch 2	~18 GB	
U-MV-small, 1024², batch 2	~16 GB	
U-MV-base, 1024², batch 2	~21 GB	

`--amp` (automatic mixed precision) roughly halves activation memory at no accuracy cost for these models. RAM: keep `num_workers × tile size` in mind; 8 workers on 1024² RGB tiles use ~4 GB.

Pitfall (WSL2). When GPU memory is exhausted, the NVIDIA driver may fall back to system memory and freeze WSL2. In the NVIDIA Control Panel set *CUDA – Sysmem Fallback Policy* to *Prefer No Sysmem Fallback*, and reduce batch size or use `--amp` instead of relying on fallback.

3 Anatomy of MMSegmentation

3.1 Two copies of the library

When MMSegmentation is installed with `pip install mmsegmentation==1.2.2`, the Python package `mmseg` is installed together with a hidden folder `mmseg/.mim` that carries the configuration files and the tools of the repository. A source checkout (`git clone https://github.com/open-mmlab/mmsegmentation`) has the same content at the top level. The two are interchangeable; the team image uses the pip package, and this repository vendors `tools/train.py` and `tools/test.py` so that the source checkout is not needed.

```
mmsegmentation/                (source checkout)      pip package equivalent
├── configs/                   experiment configs      mmseg/.mim/configs/
├── mmseg/                     the library             mmseg/
├── tools/                     command-line scripts    mmseg/.mim/tools/
├── demo/                      inference demos, notebooks
├── docs/                      documentation sources
├── tests/                     unit tests
├── projects/                  community contributions (not part of the package)
├── requirements/              dependency lists
└── model-index.yml            model zoo index used by `mim download`
```

3.2 `configs/`: the experiments

```
configs/
├── _base_/
│   ├── datasets/     one file per dataset: pipelines + dataloaders + evaluator (ade20k.py, potsdam.py, loveda.p
y, ...)
│   ├── models/       one file per architecture family: data_preprocessor + backbone + heads (pspnet_r50-d8.py,
upernet_r50.py, segformer_mit-b0.py, ...)
│   ├── schedules/    optimiser + learning-rate schedule + loops + hooks (schedule_20k.py ... schedule_320k.py)
│   └── default_runtime.py   logging, visualiser, checkpoint loading defaults
└── <method>/          full experiments = combination of the four bases + overrides, e.g.
                       segformer/segformer_mit-b2_8xb2-160k_ade20k-512x512.py
```

The file name of a full experiment encodes model, batch (`8xb2` = 8 GPUs × 2), iterations, dataset and crop size. The `_base_` folder is the part you reuse: chapter 4 shows how a project config points to `mmseg::_base_/models/upernet_r50.py` and changes only what differs.

3.3 `mmseg/`: the library

Sub-package	Content	You touch it when
`apis/`	`init_model`, `inference_model`, `show_result_pyplot`, `MMSegInferencer`, `RSInferencer` (remote-sensing sliding-window inference)	writing prediction scripts
`datasets/`	`BaseSegDataset` and one class per public dataset (`ADE20KDataset`, `PotsdamDataset`, `LoveDADataset`, `ISPRSDataset`, ...); `transforms/` with loading (`LoadImageFromFile`, `LoadAnnotations`, `LoadSingleRSImageFromFile`), augmentation (`RandomCrop`, `RandomFlip`, `PhotoMetricDistortion`, `RandomRotate`, `Albu`, ...) and formatting (`PackSegInputs`)	defining a dataset, changing augmentation
`models/backbones/`	encoders: `ResNetV1c`, `HRNet`, `UNet`, `MixVisionTransformer` (SegFormer), `SwinTransformer`, `BEiT`, `MAE`, `MSCAN` (SegNeXt), `TIMMBackbone` (any timm model), ...	choosing/adding an encoder
`models/decode_heads/`	decoders: `FCNHead`, `PSPHead`, `ASPPHead`, `DepthwiseSeparableASPPHead` (DeepLabV3+), `UPerHead`, `SegformerHead`, `Mask2FormerHead`, ...	choosing/adding a decoder
`models/necks/`	optional feature adapters between backbone and head (`FPN`, `MultiLevelNeck`, `JPU`)	rarely

Sub-package	Content	You touch it when
`models/losses/`	`CrossEntropyLoss`, `DiceLoss`, `LovaszLoss`, `FocalLoss`, `TverskyLoss`, `BoundaryLoss`	class imbalance, boundary quality
`models/segmentors/`	`EncoderDecoder` (the standard wrapper: backbone → neck → head; `whole` or `slide` inference), `CascadeEncoderDecoder`, `SegTTAModel`	test-time behaviour
`models/data_preprocessor.py`	`SegDataPreProcessor`: normalisation, BGR→RGB, padding, batching on the GPU	multispectral inputs, tile sizes
`engine/`	hooks (`SegVisualizationHook`), optimiser constructors (layer-wise decay for transformers)	monitoring, fine-tuning schedules
`evaluation/metrics/`	`IoUMetric` (IoU, Dice/F-score, precision, recall, accuracy per class), `CityscapesMetric`, `DepthMetric`	reporting
`structures/`	`SegDataSample`: the container that carries an image's ground truth, prediction and metadata through the pipeline	writing custom heads or inference code
`registry/`	the registries: `MODELS`, `DATASETS`, `TRANSFORMS`, `METRICS`, `HOOKS`, ...	registering custom components
`visualization/`	`SegLocalVisualizer`: overlays for logs and demos	
`utils/`	class names and palettes of public datasets, misc helpers	

3.4 `tools/`: the command line

Script	Purpose
`tools/train.py <config>`	train; `--work-dir`, `--resume`, `--amp`, `--cfg-options`
`tools/test.py <config> <checkpoint>`	evaluate; `--out` (save predictions), `--show-dir` (overlays), `--tta`
`tools/analysis_tools/get_flops.py <config> --shape H W`	parameters and FLOPs
`tools/analysis_tools/benchmark.py <config> <ckpt>`	inference speed
`tools/analysis_tools/browse_dataset.py <config> --output-dir DIR`	render augmented training samples (verify the pipeline)
`tools/analysis_tools/confusion_matrix.py <config> <pred dir> <save dir>`	confusion matrix from `test.py --out`
`tools/analysis_tools/analyze_logs.py <json log> --keys mIoU`	plot curves from a run's JSON log
`tools/misc/print_config.py <config>`	print the fully resolved config
`tools/misc/publish_model.py in.pth out.pth`	strip optimiser state, add hash, for release
`tools/dataset_converters/*.py`	prepare public datasets (Potsdam, Vaihingen, LoveDA, iSAID, ...)
`tools/model_converters/*.py`	convert third-party pretrained weights (Swin, MiT, BEiT, ...) to MMSegmentation keys

The team repository adds `tools/check_dataset.py`, `make_tiles.py`, `band_statistics.py`, `compare_runs.py`, `adapt_first_conv.py`, `download_zoo_weights.py`, `geospatial_inference.py`, `batch_geospatial_inference.py`, `inspect_checkpoint.py`, `verify_install.py`.

3.5 How a training run flows

```
config file  ──►  Runner (MMEngine)
                    ├── build train_dataloader
                    │     dataset (BaseSegDataset: lists img/mask pairs)
                    │       └── pipeline: Load → Augment → PackSegInputs   (CPU workers)
                    │     sampler (InfiniteSampler) + collate  ──► batch
```

```
                    ├── model.data_preprocessor: to GPU, BGR→RGB, normalise, pad, stack
                    ├── model(inputs, data_samples, mode='loss')
                    │     backbone → [neck] → decode_head (+ auxiliary_head) → losses
                    ├── optim_wrapper: backward, clip_grad, optimizer.step, [AMP]
                    ├── param_scheduler: learning rate for the next iteration
                    ├── hooks: IterTimer, Logger, Checkpoint, SegVisualization, ...
                    └── every val_interval: ValLoop
                          val_dataloader → model(mode='predict') → IoUMetric → mIoU, mFscore, ...
```

Three consequences that explain most surprises:

1 Augmentation happens in CPU worker processes (`num_workers`); slow training with an idle GPU usually means too few workers or a network drive.

2 Normalisation is **not** in the pipeline but in `data_preprocessor`, on the GPU; `mean`/`std` must have one entry per input band.

3 Metrics are computed by the evaluator on predictions resized to the original image size; test-time `mode='slide'` versus `'whole'` changes accuracy on large tiles.

3.6 Where the team's code plugs in

The `umv` package registers additional components in the same registries:

Registry	Component	Registered name
MODELS	MambaVision encoder wrapper	`MambaVisionBackbone`
MODELS	light U-Net decoder	`GenericUNetHead`
DATASETS	Acacia dataset	`UAVAcaciaDataset`
TRANSFORMS	rasterio multi-band loader	`LoadRasterioImage`

A config activates them with `custom_imports = dict(imports=['umv'])`. This is the pattern for any custom component: a Python package, a decorator (`@MODELS.register_module()`), and one line in the config.

3.7 Exercise 2

Open `mmseg/.mim/configs/_base_/models/deeplabv3plus_r50-d8.py` (path printed by `python -c "import mmseg, os; print(os.path.dirname(mmseg.__file__))"`). Identify: the backbone class and its `out_indices`; which feature level the decode head consumes (`in_index`); which level the auxiliary head consumes; and the two losses. Then find the same information in the resolved config of the Acacia model with `python tools/misc/print_config.py configs/mambavision/U-MV-small.py`.

4 Configuration files

4.1 A config is a Python file that builds a dictionary

MMEngine executes the config file and keeps every top-level variable. Four top-level names matter to the `Runner`: `model`, the three dataloaders with their evaluators, the schedule (`optim_wrapper`, `param_scheduler`, `train_cfg`, `val_cfg`, `test_cfg`) and the runtime (`default_hooks`, `env_cfg`, `visualizer`, `log_level`, `load_from`, `resume`, `randomness`). Everything else (`crop_size`, `norm_cfg`, `metainfo`) is a helper variable.

Every component is a `dict` with a `type` key naming a registered class; the remaining keys are its constructor arguments:

```
decode_head = dict(
    type='UPerHead',              # class registered in MODELS
    in_channels=[256, 512, 1024, 2048],
    in_index=[0, 1, 2, 3],
    channels=512,
    num_classes=2,
    loss_decode=dict(type='CrossEntropyLoss', use_sigmoid=False, loss_weight=1.0))
```

`MODELS.build(decode_head)` therefore equals `UPerHead(in_channels=..., ...)`. The constructor signature is the documentation: `python -c "import inspect; from mmseg.models.decode_heads import UPerHead; print(inspect.signature(UPerHead.__init__))"`.

4.2 Inheritance with `_base_`

A config lists parent files in `_base_`; their dictionaries are merged, and the child's values override the parents' key by key (deep merge):

```
_base_ = [
    'mmseg::_base_/models/upernet_r50.py',      # from the installed package
    './_base_/dataset.py',                      # this project's data
    './_base_/schedule.py',                     # this project's schedule
    'mmseg::_base_/default_runtime.py',
]
model = dict(decode_head=dict(num_classes=2), auxiliary_head=dict(num_classes=2))
```

Only `num_classes` changes; the backbone, channels and losses come from the base. The `mmseg::` prefix resolves inside the installed package, so nothing is copied from upstream and the base is always the version-matched one.

Rules of merging:

Situation	Behaviour	Remedy
child key exists in base	value replaced (dicts are merged recursively)	—
replace a whole dict instead of merging	add `_delete_=True` inside the child dict	e.g. a different optimiser: `optim_wrapper = dict(_delete_=True, type='OptimWrapper', optimizer=dict(type='SGD', lr=0.01))`
lists (`_base_` pipelines, `param_scheduler`, `loss_decode`)	replaced as a whole, never merged	re-state the full list
same top-level variable defined in two bases	error "Duplicate key is not allowed among bases"	keep helper variables (e.g. `crop_size`) in one base only

4.3 Reusing base values: `{{_base_.name}}`

A child may reference a variable of a base file with `{{_base_.name}}`. The value is substituted **after** the child is executed, so it can be used as a whole value but not in arithmetic:

```
crop_size = {{_base_.crop_size}}                    # OK: whole value
num_classes = {{_base_.num_classes}}                # OK
model = dict(data_preprocessor=dict(size=crop_size), test_cfg={{_base_.slide_test_cfg}})
stride = crop_size[0] // 2                          # ERROR: crop_size is still a placeholder string here
```

The project template therefore defines derived values (`num_classes`, `slide_test_cfg`) in `_base_/dataset.py` and references them whole.

4.4 Overriding from the command line

Any key can be overridden without editing files, which is how experiments are varied and how a config is pointed at another machine's data:

```
python tools/train.py projects/buildings/configs/upernet_r50.py \
    --cfg-options train_dataloader.batch_size=2 optim_wrapper.optimizer.lr=5e-5 \
                  train_dataloader.dataset.data_root=/data/buildings_v2 \
                  randomness.seed=1
```

Nested keys use dots; lists use `key="[a,b]"`. The overridden config is what gets saved in the work directory.

4.5 Registries and scopes

Registries are name → class tables, one per component kind (`MODELS`, `DATASETS`, `TRANSFORMS`, `METRICS`, `HOOKS`, `OPTIM_WRAPPERS`, ...). MMEngine owns the root registries; MMSegmentation's registries are children with scope `mmseg`; MMDetection's have scope `mmdet`. When a name exists in several scopes (e.g. `ResNet` in mmseg, mmdet and mmpretrain) prefix it: `type='mmdet.ResNet'`. `default_scope = 'mmseg'` in `default_runtime.py` sets the default.

Custom components are registered by importing the module that decorates them:

```
custom_imports = dict(imports=['umv'], allow_failed_imports=False)
```

`allow_failed_imports=False` turns a typo into an immediate error rather than a mysterious "X is not in the registry" later.

4.6 Inspecting a config

```
python tools/misc/print_config.py projects/buildings/configs/upernet_r50.py      # resolved, all bases merged
python tools/misc/print_config.py ... --cfg-options model.decode_head.num_classes=3
```

Every run also writes the resolved config into its work directory (`work_dirs/<run>/<timestamp>/vis_data/config.py` and next to the log), which is the file to archive with a result: it records exactly what was run, whatever the `_base_` files contained at the time.

4.7 Reading an unfamiliar config: a checklist

1. `model.type` and `model.backbone.type` — which architecture?
2. `model.backbone.in_channels` and `data_preprocessor.mean/std` — how many bands, which normalisation, `bgr_to_rgb`?
3. `decode_head.num_classes` (and `auxiliary_head.num_classes`) — do they equal the length of `metainfo['classes']`?
4. `train_pipeline` — crop size, augmentations; `test_pipeline` — resize?
5. `train_cfg.max_iters`, `val_interval`, `optimizer.lr`, `param_scheduler`.
6. `model.test_cfg.mode` — `whole` or `slide` and with which window.
7. `load_from` / `backbone.init_cfg` / `model.pretrained` — where weights come from.

8 `default_hooks.checkpoint.save_best` — which checkpoint is "best".

4.8 Pitfalls

- `SyncBN` (the zoo default) requires distributed training; on one GPU set `norm_cfg = dict(type='BN', requires_grad=True)` and pass it to backbone and heads, as the templates do.
- `num_classes` is counted **including** background; a binary task has 2.
- `data_preprocessor.size` pads to that size; a crop smaller than the size is padded with `pad_val` (image) and `seg_pad_val=255` (mask).
- A `_base_` path is relative to the file that names it, not to the working directory.

4.9 Exercise 3

Take `tutorial/templates/project/configs/upernet_r50.py` and, using only `--cfg-options`, (a) halve the learning rate, (b) change the crop to 768 × 768 (which keys must change together?), (c) disable the auxiliary head. Verify each with `print_config.py`.

5 From GIS layers to training tiles

5.1 What the model needs

Item	Requirement	Why
Images	georeferenced rasters of fixed tile size; 8-bit 3-band for RGB workflows, any dtype/band count with the rasterio loader	fixed size gives regular batches; georeferencing lets predictions return to GIS
Masks	single-band 8-bit rasters on the **same grid** as the images; pixel value = class index (0 = background); 255 = ignore	the loss compares pixel by pixel
Names	identical base names in `img_dir/<split>` and `ann_dir/<split>`	this is how pairs are matched
Splits	`train`, `val` (model selection), `test` (final report), optionally an out-of-distribution set	the test set must never influence choices

Tile size is a design decision, not a default. The tile must contain the object with its context at the working GSD: a 5 m crown at 2.5 cm is 200 px, and its shadow and neighbours matter, hence 1024 px for the Acacia project; buildings at 10 cm fit in 512 px; a 10 m Sentinel-2 field pattern fits in 256 px. Larger tiles cost memory quadratically.

5.2 The reference workflow (ArcGIS Pro)

1. **Reference layer.** Verified polygons with an integer class field (`class_id`: 1 = target, 2 = second class, ...). Areas not interpreted are left outside the polygons and outside an *interpretation extent* polygon.
2. **Rasterise.** *Polygon to Raster*: value field `class_id`, cell size and snap raster = the orthomosaic, `NoData` → 0 with *Reclassify*, and 255 outside the interpretation extent (mask with *Extract by Mask* then *Con(IsNull(x), 255, x)*). Export as 8-bit unsigned GeoTIFF.
3. **Tile.** Either *Export Training Data For Deep Learning* (Classified Tiles, tile size, stride, no rotation) or the team tool below.
4. **Check.** `python tools/check_dataset.py /data/<project>`.

The same is achieved with GDAL: `gdal_rasterize -a class_id -tr <res> <res> -te <extent> -ot Byte -a_nodata 255 polygons.gpkg labels.tif`.

5.3 Tiling with `tools/make_tiles.py`

```
python tools/make_tiles.py --image /data/raw/ortho_block12.tif --mask /data/raw/labels_block12.tif \
    --out /data/buildings --tile 512 --overlap 0 --split-blocks 4 --val-frac 0.15 --test-frac 0.15 \
    --min-labeled 0.5 --min-target 50 --keep-empty 0.3 --prefix b12_
```

What it does, and why:

- **Spatial-block split.** The raster is divided into 4 × 4 blocks and whole blocks are assigned to train/val/test. Randomly assigning tiles would put near-identical neighbours into train and test and inflate the test score; the Acacia study used spatially distinct regions for the same reason.
- **Empty-tile control.** Tiles without any target pixel are kept with probability 0.3, which keeps enough pure background to learn the negatives without drowning the positives.
- **Ignore handling.** Tiles with less than 50 % interpreted pixels are skipped; 255 pixels inside kept tiles remain 255 and are ignored by the loss.
- **Provenance.** `tiles_index.csv` records split, block, offsets and class pixel counts per tile; it is the file from which class frequencies and weights are computed.

Repeat per orthomosaic with a different `--prefix`; the folder structure is shared. Multiple regions: keep at least one whole region out of training as the generalisability set, exactly as in the paper.

5.4 Checking the dataset

```
python tools/check_dataset.py /data/buildings --splits train val test
```

reports pair counts, sizes, dtypes, band counts and mask values per split, and side-car files (`*.aux.xml`, `*.ovr`) that ArcGIS leaves behind. Then render a few augmented samples to see what the network will see:

```
python tools/analysis_tools/browse_dataset.py projects/buildings/configs/upernet_r50.py --output-dir work_dirs/b
rowse --not-show
```

Checkpoint. `RESULT: OK`; mask values are exactly the class indices plus 255; the overlays in `work_dirs/browse` show the mask on the correct objects (a shifted mask means a grid mismatch between image and label rasters).

5.5 Class statistics and imbalance

From `tiles_index.csv` (pandas):

```
import pandas as pd
t = pd.read_csv('/data/buildings/tiles_index.csv')
cols = [c for c in t.columns if c.startswith('class_')]
freq = t.loc[t.split == 'train', cols].sum(); print((freq / freq.sum()).round(4))
```

A target class below ~5 % of pixels calls for a region-based loss (Dice, Lovasz) in addition to cross-entropy, background-tile down-sampling at tiling time, and `cat_max_ratio` in `RandomCrop` (rejects crops dominated by one class). See chapter 7.4 for the losses.

5.6 Multispectral and 16-bit data

Keep the native radiometry in the tiles (uint16 reflectance × 10 000 for Sentinel-2, 12-bit for many UAV multispectral sensors); scaling to reflectance is done at load time (chapter 6.6). Record per-band statistics once:

```
python tools/band_statistics.py /data/lulc_s2/img_dir/train --bands 1 2 3 4 5 6 7 8 9 10 --scale 1e-4
```

and copy the printed `mean`/`std` into the config's `data_preprocessor`.

5.7 Pitfalls

- Masks saved as RGB or as 0/255 instead of 0/1: `check_dataset.py` reports `mask values=[0, 255]` or `mask bands=3`; re-export as single-band index.
- Compressed masks with a colour table are fine; masks with a NoData value of 0 are not (0 is background, not NoData) — set NoData to none or 255.
- JPEG-compressed image tiles lose fine texture; use deflate/LZW.
- Tiles that straddle the orthomosaic edge contain black (0) fill; either skip them (`--min-labeled`) or mark the fill as 255 in the mask.

5.8 Exercise 4

Tile a small orthomosaic with `--split-blocks 2` and `--split-blocks 8`. How does the test set differ, and which one gives an estimate closer to performance on a new region?

6 Datasets, pipelines and augmentation

6.1 The dataset object

`BaseSegDataset` lists image–mask pairs and applies a pipeline to each. The generic class is sufficient for most projects; the class names and colours are passed through `metainfo`:

```
dataset = dict(
    type='BaseSegDataset',
    data_root='/data/buildings',
    data_prefix=dict(img_path='img_dir/train', seg_map_path='ann_dir/train'),
    img_suffix='.tif', seg_map_suffix='.tif',
    metainfo=dict(classes=('background', 'building'), palette=[[0, 0, 0], [220, 60, 30]]),
    reduce_zero_label=False,
    pipeline=train_pipeline)
```

Arguments that matter:

Argument	Meaning
`data_root`, `data_prefix`	folders; pairs are matched by base name: `img_dir/train/x.tif` ↔ `ann_dir/train/x.tif`
`img_suffix`, `seg_map_suffix`	file endings; MMSegmentation's defaults are `.jpg`/`.png`, hence the explicit `.tif`
`metainfo`	`classes` (index order) and `palette` (display colours); written into checkpoints, used by metrics and visualisation
`reduce_zero_label`	if `True`, label 0 becomes 255 (ignored) and every other label is decremented; used by datasets whose 0 means "unlabeled" (ADE20K, Potsdam). Keep `False` when 0 is a real background class
`ignore_index`	label excluded from loss and metrics (255)
`ann_file`	optional text file listing base names, to define a split without moving files
`indices`	use only the first N (or a list of) samples, for quick debugging runs

A permanent dataset deserves a class (`umv/datasets/uav_acacia.py`): it fixes suffixes and classes so that every config is shorter and consistent. The class is 15 lines: subclass `BaseSegDataset`, set `METAINFO`, set the suffix defaults in `__init__`, decorate with `@DATASETS.register_module()`.

6.2 The pipeline: loading

A pipeline is a list of transforms applied in order to a `results` dictionary that starts as `{'img_path': ..., 'seg_map_path': ...}`.

Transform	What it loads	Use
`LoadImageFromFile`	8-bit 1- or 3-band image via OpenCV, **BGR** order, uint8	RGB tiles (jpg, png, tif)
`LoadAnnotations(reduce_zero_label=False)`	single-band mask via Pillow, uint8	always, after the image loader (in the test pipeline after `Resize`, so that the mask is not resized)
`LoadSingleRSImageFromFile(to_float32=True)`	any GeoTIFF via GDAL, all bands, file order, float32	multispectral, when GDAL Python bindings exist
`LoadMultipleRSImageFromFile`	two rasters (`img_path`, `img_path2`)	change detection
`LoadRasterioImage(bands=..., scale=..., nodata_fill=..., clip=...)` (team)	selected bands via rasterio, float32, scaled	RGB+NIR, multispectral, 16-bit, SAR
`LoadImageFromNDArray`	an in-memory array	inference from Python

The BGR order of `LoadImageFromFile` is the reason for `bgr_to_rgb=True` in every RGB config: the preprocessor converts to RGB before normalising with the ImageNet mean `[123.675, 116.28, 103.53]`, which is in RGB order. Loaders that return bands in the requested order (`LoadRasterioImage`) must be paired with `bgr_to_rgb=False`.

6.3 The pipeline: augmentation

Verified signatures (MMSegmentation 1.2.2):

Transform	Arguments	Effect
`RandomResize`	`scale=(W, H)`, `ratio_range=(0.5, 2.0)`, `keep_ratio=True`	scale the tile by a random factor: the network sees objects at several sizes
`RandomCrop`	`crop_size=(H, W)`, `cat_max_ratio=0.75`, `ignore_index=255`	take a crop; reject crops where one class exceeds 75 % of pixels (up to 10 tries)
`RandomFlip`	`prob=0.5`, `direction='horizontal'` or `'vertical'`	for nadir imagery both directions are valid
`RandomRotate`	`prob`, `degree=30`, `pad_val=0`, `seg_pad_val=255`	rotation with padding; objects without preferred orientation (roads, fields)
`PhotoMetricDistortion`	`brightness_delta=32`, `contrast_range=(0.5,1.5)`, `saturation_range=(0.5,1.5)`, `hue_delta=18`	radiometric jitter; **8-bit 3-band only**
`RandomRotFlip`	`rotate_prob`, `flip_prob`, `degree=(-20, 20)`	combined
`RandomMosaic`	`prob`, `img_scale=(640, 640)`	four tiles in one; needs `MultiImageMixDataset`
`RandomCutOut`	`prob`, `n_holes`, `cutout_shape` or `cutout_ratio`	erases patches (regularisation)
`Rerange`	`min_value=0`, `max_value=255`	linear rescale to a range
`CLAHE`	`clip_limit=40.0`, `tile_grid_size=(8, 8)`	contrast equalisation (8-bit)
`AdjustGamma`	`gamma=1.0`	gamma (8-bit)
`Resize`	`scale=(W, H)`, `keep_ratio=True`	deterministic resize (test pipeline)
`ResizeToMultiple`	`size_divisor=32`	pad-free sizing for transformers
`RGB2Gray`	`out_channels`, `weights`	grayscale
`Albu`	`transforms=[...]` (Albumentations)	any Albumentations transform on 8-bit RGB
`GenerateEdge`	`edge_width=3`	boundary maps for edge-aware losses
`PackSegInputs`	`meta_keys=(...)`	final step: `inputs` (C×H×W tensor) + `data_samples` (`SegDataSample` with `gt_sem_seg` and metadata)

Reference training pipeline for 8-bit RGB (the Acacia recipe):

```
train_pipeline = [
    dict(type='LoadImageFromFile'),
    dict(type='LoadAnnotations', reduce_zero_label=False),
    dict(type='RandomResize', scale=(1024, 1024), ratio_range=(0.5, 2.0), keep_ratio=True),
    dict(type='RandomCrop', crop_size=(1024, 1024), cat_max_ratio=0.75),
    dict(type='RandomFlip', prob=0.5),
    dict(type='PhotoMetricDistortion'),
    dict(type='PackSegInputs'),
]
test_pipeline = [
    dict(type='LoadImageFromFile'),
```

```
    dict(type='Resize', scale=(1024, 1024), keep_ratio=True),
    dict(type='LoadAnnotations', reduce_zero_label=False),
    dict(type='PackSegInputs'),
]
```

Choosing augmentations is a statement about invariances. Nadir aerial imagery is invariant to flips in both axes and to rotation; oblique imagery is not. Scale jitter should cover the GSD range at which the model will be applied (0.5–2.0 covers 1.25–5 cm around a 2.5 cm training GSD). Radiometric jitter stands in for illumination and sensor differences; it does not replace training data from other seasons.

6.4 The data preprocessor

`SegDataPreProcessor` runs on the GPU on each batch: `bgr_to_rgb`, then `(x - mean) / std` per channel, then padding to `size` (or to a multiple of `size_divisor`) with `pad_val` for images and `seg_pad_val=255` for masks, and stacking. `mean` and `std` have one entry per input channel; the ImageNet values are correct for 8-bit RGB with an ImageNet-pretrained encoder, and dataset statistics (`tools/band_statistics.py`) for anything else.

6.5 Dataloaders

```
train_dataloader = dict(
    batch_size=4, num_workers=8, persistent_workers=True,
    sampler=dict(type='InfiniteSampler', shuffle=True),
    dataset=dict(...))
val_dataloader = dict(batch_size=2, num_workers=4, persistent_workers=True,
    sampler=dict(type='DefaultSampler', shuffle=False), dataset=dict(...))
```

`InfiniteSampler` draws batches forever (iteration-based training); `DefaultSampler` makes one pass (evaluation). `num_workers` are CPU processes running the pipeline; start at 8 and raise it while `data_time` in the log is larger than ~10 % of `time`. `batch_size` is per GPU. Oversampling a small dataset: wrap it as `dict(type='RepeatDataset', times=5, dataset=dict(...))`; combining datasets: `dict(type='ConcatDataset', datasets=[...])`.

6.6 Multi-band and non-8-bit inputs

Three changes turn the RGB recipe into a multispectral one:

```
bands = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]                    # 1-based, output order
train_pipeline = [
    dict(type='LoadRasterioImage', bands=bands, scale=1 / 10000.0, clip=(0.0, 1.5)),   # 1. loader
    dict(type='LoadAnnotations', reduce_zero_label=False),
    dict(type='RandomResize', scale=(256, 256), ratio_range=(0.75, 1.5), keep_ratio=True),
    dict(type='RandomCrop', crop_size=(256, 256), cat_max_ratio=0.9),
    dict(type='RandomFlip', prob=0.5),
    dict(type='RandomFlip', prob=0.5, direction='vertical'),
    # 2. no PhotoMetricDistortion / CLAHE / AdjustGamma (8-bit BGR only)
    dict(type='PackSegInputs'),
]
data_preprocessor = dict(type='SegDataPreProcessor', size=(256, 256),
                         mean=[...10 values...], std=[...10 values...], bgr_to_rgb=False)   # 3. per band
model = dict(backbone=dict(in_channels=10), ...)
```

Variants:

Input	Loader settings	mean/std
RGB 8-bit (default)	`LoadImageFromFile`, `bgr_to_rgb=True`	ImageNet
RGB+NIR 8-bit (4 bands)	`LoadRasterioImage(bands=[1,2,3,4])`, `bgr_to_rgb=False`	ImageNet values + a NIR mean/std from `band_statistics.py`, or all four from statistics

Input	Loader settings	mean/std
Sentinel-2 L2A uint16	`LoadRasterioImage(bands=..., scale=1e-4, clip=(0, 1.5))`	statistics in reflectance
UAV multispectral 12-bit	`scale=1/4095`	statistics
SAR dB float32	`clip=(-30, 5)`, optional `offset`	statistics
GDAL loader	`LoadSingleRSImageFromFile` reads all bands in file order as float32, no scaling: add `scale` via the preprocessor `std` (e.g. `std=[10000*s ...]`) or pre-scale the tiles	

Pretrained encoders expect 3 channels. Options for `in_channels != 3`: (a) adapt the first convolution of an ImageNet checkpoint with `tools/adapt_first_conv.py` and load it with `init_cfg=dict(type='Pretrained', checkpoint=...)`; (b) train the stem from scratch (`pretrained=None`); (c) keep the RGB bands in the pretrained stem and feed extra bands to a second small stem (custom backbone; beyond this tutorial). Option (a) is the default choice and preserves most of the pretraining benefit.

Pitfall. `SegVisualizationHook` and `browse_dataset.py` render three channels as RGB; with 10-band inputs set `default_hooks.visualization.draw=False` or expect meaningless images.

6.7 Verifying a pipeline in Python

```
from mmengine.config import Config
from mmseg.registry import DATASETS
from mmseg.utils import register_all_modules
register_all_modules(init_default_scope=True)
cfg = Config.fromfile('projects/lulc/configs/upernet_r50_10band.py')
ds = DATASETS.build(cfg.train_dataloader.dataset)
item = ds[0]
print(item['inputs'].shape, item['inputs'].dtype)            # torch.Size([10, 256, 256]) torch.float32
print(item['data_samples'].gt_sem_seg.data.unique())         # class indices present (+255)
print(item['data_samples'].metainfo)                         # img_path, ori_shape, flip, ...
```

Ten lines that answer most "why does my model not learn" questions: wrong band order, wrong scaling, masks with unexpected values, or a crop that does not contain the target.

6.8 Exercise 5

Write the training pipeline and preprocessor for a 4-band (R, G, B, NIR) 8-bit UAV dataset stored as B, G, R, NIR in the file, keeping the ImageNet normalisation for the three visible bands. Which `bands` order and which `bgr_to_rgb` value are correct?

7 Models: choosing, adapting and switching architectures

7.1 A map of the zoo

Family	Backbone → head	Config base (`mmseg::_base_/models/`)	Pretrained encoder	Character
U-Net	`UNet` → `FCNHead`	`fcn_unet_s5-d16.py`	none (from scratch)	small, sharp boundaries, needs more iterations; good for few classes and thin objects
PSPNet	`ResNetV1c` → `PSPHead`	`pspnet_r50-d8.py`	ImageNet (`open-mmlab://resnet50_v1c`)	pyramid pooling for context
DeepLabV3+	`ResNetV1c` → `DepthwiseSeparableASPPHead`	`deeplabv3plus_r50-d8.py`	ImageNet	atrous context + low-level skip; the classic strong CNN baseline
UPerNet	`ResNetV1c` / `SwinTransformer` / `ConvNeXt` → `UPerHead`	`upernet_r50.py`, `upernet_swin.py`, `upernet_convnext.py`	ImageNet	feature-pyramid fusion; the standard head for transformer encoders
SegFormer	`MixVisionTransformer` → `SegformerHead`	`segformer_mit-b0.py` (+ B1–B5 overrides)	ImageNet (MiT)	light transformer, efficient at 1024²; competitive in the Acacia study
SegNeXt	`MSCAN` → `LightHamHead`	`segnext_mscan-t` configs	ImageNet	convolutional attention, efficient
Mask2Former	any → `Mask2FormerHead`	`mask2former` configs (needs MMDetection)	ImageNet	mask-classification decoder; strong but heavy, slow to train
Segmenter	`VisionTransformer` → `SegmenterMaskTransformerHead`	`segmenter_vit-b16_mask.py`	ImageNet	pure ViT
U-MV (team)	`MambaVisionBackbone` → `GenericUNetHead`	`configs/_base_/models/umv_unet.py` (this repo)	ImageNet (HF Hub)	hybrid Mamba–transformer encoder, light decoder; best accuracy/cost in the Acacia study

Guidance from the Acacia benchmark (test mIoU): U-MV 85.3–85.4, SegFormer-B2 85.2, Mask2Former-R50 84.9, U-Net-R50 84.2, PSPNet-R50 81.9, DeepLabV3+-R50 81.1. Transformers and hybrids generalised better to the unseen region. Start a new project with two baselines from different families (e.g. DeepLabV3+ and SegFormer-B2) plus U-MV, and let the comparison decide.

7.2 Adapting a zoo model to your data

Every adaptation is the same four edits, shown for UPerNet-R50:

```
_base_ = ['mmseg::_base_/models/upernet_r50.py', './_base_/dataset.py', './_base_/schedule.py',
          'mmseg::_base_/default_runtime.py']
crop_size = {{_base_.crop_size}}
num_classes = {{_base_.num_classes}}
norm_cfg = dict(type='BN', requires_grad=True)            # 1. single-GPU normalisation
model = dict(
    data_preprocessor=dict(size=crop_size),               # 2. tile size (mean/std/bgr_to_rgb for non-RGB)
    backbone=dict(norm_cfg=norm_cfg),                     #    (+ in_channels for multispectral)
    decode_head=dict(num_classes=num_classes, norm_cfg=norm_cfg),      # 3. classes
```

```
    auxiliary_head=dict(num_classes=num_classes, norm_cfg=norm_cfg),
    test_cfg={{_base_.slide_test_cfg}})                 # 4. inference mode
```

The auxiliary head is a second, shallow classifier on an intermediate feature level used only during training (loss weight 0.4); it stabilises deep CNNs and is absent from SegFormer and U-MV. Set `auxiliary_head=None` to drop it.

7.3 Pretrained weights

Mechanism	Where	Applies to
`model.pretrained='open-mmlab://resnet50_v1c'` or a URL/path	zoo CNN bases	whole backbone, loaded at `init_weights()`
`backbone.init_cfg=dict(type='Pretrained', checkpoint=URL_or_path)`	transformer bases (MiT, Swin, ConvNeXt)	backbone
`load_from='work_dirs/.../best_mIoU_iter_x.pth'`	top level	whole segmentor; fine-tune a previous run on new data (classes must match, or set `strict=False` semantics by deleting the head keys)
`resume=True` / `--resume`	top level	continue the same run (weights + optimiser + iteration)
`MambaVisionBackbone(pretrained=True)`	team backbone	ImageNet weights from the Hugging Face Hub

Download once for offline hosts: `python tools/download_zoo_weights.py` (ResNet-50/101 V1c, MiT-B1/B2, Swin-T) and point the config to the local file. Encoder weights for SegFormer/Swin come converted to MMSegmentation key names from `download.openmmlab.com`; weights from timm or torchvision require the converters in `tools/model_converters/`.

7.4 Losses for imbalance and boundaries

```
loss_decode=[dict(type='CrossEntropyLoss', use_sigmoid=False, loss_weight=1.0),
             dict(type='DiceLoss', loss_weight=3.0)]              # the Acacia recipe
```

Loss	When	Notes
`CrossEntropyLoss`	always, as the base term	`avg_non_ignore=True` averages over labeled pixels only
`DiceLoss`	small or sparse targets (crowns, buildings)	region overlap; weight 1–3 relative to CE
`LovaszLoss(loss_type='multi_class', reduction='none')`	thin, connected structures (roads, rivers)	direct IoU surrogate; use after a few thousand CE iterations or together with CE
`FocalLoss(use_sigmoid=True)`	many easy background pixels	binary/one-vs-rest formulation
`TverskyLoss`	trade recall against precision	`alpha`, `beta`
`BoundaryLoss`	boundary accuracy (with `GenerateEdge`)	add with small weight

Each loss needs a distinct `loss_name` only when the same type appears twice.

Pitfall (verified). `CrossEntropyLoss(class_weight=[...])` indexes the weight vector by every label value when averaging, so any 255 pixel — present whenever padding or unlabeled areas exist — raises an index error in 1.2.2. Handle imbalance with Dice/Lovasz/Focal or tiling-time down-sampling instead.

7.5 Switching between models

Because dataset and schedule are shared bases, switching architecture is switching one file:

```
for m in unet_fcn deeplabv3plus_r50 upernet_r50 segformer_b2 umv_small; do
  python tools/train.py projects/buildings/configs/$m.py --work-dir work_dirs/buildings/$m
done
python tools/compare_runs.py work_dirs/buildings          # one table, best/last/test metrics
```

`tutorial/templates/project/scripts/run_experiments.sh` does exactly this and appends the test-split evaluation. Compare at equal iterations, equal crop and equal augmentation; report parameters and FLOPs (`tools/analysis_tools/get_flops.py <config> --shape 1024 1024`) next to accuracy, as in the paper's efficiency figure.

7.6 Size and speed

Measured with `mmengine.analysis.get_model_complexity_info` on the template configs (2 classes; FLOPs at 512² and scaled to 1024²):

Model	Params	FLOPs @512²	FLOPs @1024²	Notes
SegFormer-B1	13.7 M	15 G	0.06 T	fastest transformer
SegFormer-B2	24.7 M	25 G	0.10 T	best accuracy/cost among the zoo transformers in the Acacia study
U-Net (s5-d16)	29.1 M	203 G	0.81 T	full-resolution decoder is costly
U-MV-tiny	35.4 M	—	0.14 T	paper; 9.3 h / 100k it. on TITAN RTX
DeepLabV3+-R50 (d8)	43.6 M	176 G	0.71 T	output stride 8 is expensive at 1024²
U-MV-small	54.2 M	—	0.22 T	paper; 11.8 h
UPerNet-R50	66.4 M	237 G	0.95 T	
U-MV-base	—	—	0.40 T	paper; 18.1 h

FLOPs scale linearly with pixels for CNNs and roughly so for the windowed and reduced-attention transformers used here; a 1024² tile costs about 4× a 512² tile.

7.7 Adding a custom architecture

The team backbone is the worked example (`umv/models/mamba_vision.py`):

1. subclass `mmengine.model.BaseModule`, implement `forward(x)` returning a list of feature maps at strides 4/8/16/32;
2. decorate with `@MODELS.register_module()`;
3. expose it with `custom_imports = dict(imports=['umv'])`;
4. tell the head the channel widths (`in_channels=[...]`).

A custom head subclasses `BaseDecodeHead`, implements `forward(inputs)` returning logits at stride 4 and inherits loss computation, resizing and prediction (`umv/models/unet_head.py`, 90 lines).

7.8 Exercise 6

Create `projects/<yours>/configs/segformer_b1.py` from `segformer_b2.py` (MiT-B1: `embed_dims=64`, `num_layers=[2, 2, 2, 2]`, checkpoint `mit_b1`). Compare parameters and FLOPs of B1, B2 and DeepLabV3+-R50 with `get_flops.py` at 512 × 512.

8 Training

8.1 Starting a run

```
python tools/train.py projects/buildings/configs/upernet_r50.py --work-dir work_dirs/buildings/upernet_r50
```

Options: `--resume` (continue from the last checkpoint in the work directory), `--amp` (mixed precision), `--cfg-options key=value ...`. Without `--work-dir` the run goes to `work_dirs/<config name>/`. Each run creates a timestamped sub-folder with the log, `vis_data/scalars.json` (all logged numbers) and the resolved config.

Before a long run, a two-minute smoke test catches most configuration errors:

```
python tools/train.py <config> --work-dir work_dirs/smoke \
    --cfg-options train_cfg.max_iters=20 train_cfg.val_interval=10 \
                  train_dataloader.dataset.indices=16 val_dataloader.dataset.indices=8 \
                  default_hooks.logger.interval=1
```

Checkpoint. The log shows 20 iterations with decreasing `loss`, one validation with an mIoU table over your class names, and a checkpoint file.

8.2 The schedule

```
optimizer = dict(type='AdamW', lr=1e-4, weight_decay=0.05, betas=(0.9, 0.999))
optim_wrapper = dict(type='OptimWrapper', optimizer=optimizer,
                     clip_grad=dict(max_norm=1.0, norm_type=2),
                     paramwise_cfg=dict(custom_keys={'backbone': dict(lr_mult=0.1),
                                                     'norm': dict(decay_mult=0.0)}))
param_scheduler = [
    dict(type='LinearLR', start_factor=1e-3, by_epoch=False, begin=0, end=500),
    dict(type='PolyLR', eta_min=0, power=0.9, begin=500, end=40000, by_epoch=False)]
train_cfg = dict(type='IterBasedTrainLoop', max_iters=40000, val_interval=2000)
```

Element	Meaning	Rules of thumb
optimiser	AdamW for transformers and hybrids (lr 6e-5–1e-4); SGD momentum 0.9 (lr 0.01) is the zoo default for ResNet models and also works	keep one optimiser across the models you compare
`lr_mult` for backbone	pretrained encoder learns 10× slower than the new head	the Acacia runs: encoder 1e-5, decoder 1e-4
`decay_mult=0` for norm	no weight decay on normalisation parameters	standard
warm-up (`LinearLR`)	avoids early divergence of AdamW	500–1500 iterations
`PolyLR` power 0.9	smooth decay to 0	the segmentation default
`max_iters`	budget; iterations × batch / tiles = epochs	40k for 5–30k tiles; 100k for the paper
`val_interval`	how often the validation set is scored	5 % of `max_iters`
`clip_grad`	caps the gradient norm; stabilises small batches	1.0 (0.01 in the Acacia runs)

Batch size is limited by memory; 2–4 at 1024^2, 8 at 512^2. Since BatchNorm statistics degrade with batch 1–2, prefer `--amp` over a smaller batch, or use GroupNorm-based models (SegFormer uses LayerNorm and is insensitive).

8.3 Hooks

```
default_hooks = dict(
    timer=dict(type='IterTimerHook'),
    logger=dict(type='LoggerHook', interval=50, log_metric_by_epoch=False),
    param_scheduler=dict(type='ParamSchedulerHook'),
    checkpoint=dict(type='CheckpointHook', by_epoch=False, interval=2000, save_best='mIoU', max_keep_ckpts=3),
    sampler_seed=dict(type='DistSamplerSeedHook'),
    visualization=dict(type='SegVisualizationHook', draw=True, interval=200))
```

```
custom_hooks = [dict(type='EarlyStoppingHook', monitor='mIoU', patience=5, min_delta=0.1)]
```

`save_best='mIoU'` keeps `best_mIoU_iter_<n>.pth`, the checkpoint of record; `max_keep_ckpts` limits disk use (each checkpoint stores the optimiser state, 2–3× the model size). `SegVisualizationHook(draw=True)` writes validation overlays to `vis_data/vis_image` (RGB only). `EarlyStoppingHook` stops when validation mIoU has not improved by 0.1 for 5 validations.

8.4 Reading the log

```
Iter(train) [ 2050/40000]  base_lr: 9.6e-05 lr: 9.6e-05  eta: 3:12:41  time: 0.30  data_time: 0.01
                           memory: 9124  loss: 0.41  decode.loss_ce: 0.21  decode.loss_dice: 0.15  aux.loss_ce:
 0.05  decode.acc_seg: 96.1
Iter(val) [500/500]  aAcc: 97.2  mIoU: 78.4  mAcc: 85.1  mFscore: 87.3  mPrecision: 89.0  mRecall: 85.1
```

Field	Diagnosis
`data_time` 0.1 × `time`	data loading is the bottleneck: more `num_workers`, local SSD, smaller tiles or COG
`memory` close to the GPU limit	reduce batch or crop, use `--amp`
`loss` flat from the start	learning rate too low, wrong labels (check masks), or `num_classes` mismatch
`loss` NaN	learning rate too high, missing `clip_grad`, or NaN in 16-bit inputs (check `nodata_fill`)
`decode.acc_seg` high but `mIoU` low	class imbalance: accuracy is dominated by background
validation mIoU drops while train loss falls	over-fitting: more augmentation, earlier stop, smaller model

Curves: `tools/analysis_tools/analyze_logs.py <run>/vis_data/<timestamp>.json --keys mIoU loss`, or TensorBoard (`dict(type='TensorboardVisBackend')` in `visualizer.vis_backends`).

8.5 Training several models

```
bash tutorial/templates/project/scripts/run_experiments.sh projects/buildings deeplabv3plus_r50 segformer_b2 umv
_small
```

trains each config, evaluates it on the test split and writes `work_dirs/buildings/summary.md`. Run it inside `tmux`. Keep everything that is compared identical except the model (chapter 12 gives the record template).

8.6 Fine-tuning a trained model on new data

```
load_from = '/weights/mambavision-s_generic-unet_acacia-88/best_mIoU_iter_95000.pth'
optim_wrapper = dict(optimizer=dict(lr=2e-5))
param_scheduler = [dict(type='PolyLR', eta_min=0, power=0.9, begin=0, end=10000, by_epoch=False)]
train_cfg = dict(type='IterBasedTrainLoop', max_iters=10000, val_interval=1000)
```

Same classes: everything loads. New class count: the head's final layer has a different shape; MMEngine reports a size mismatch warning for those tensors and initialises them randomly, which is the intended behaviour for transfer.

8.7 Reproducibility

`randomness = dict(seed=0)` fixes data order and initialisation; `deterministic=True` also fixes cuDNN algorithms (slower). Two seeds bound the run-to-run variation, needed before claiming a 0.3 mIoU difference is real.

8.8 Exercise 7

Run the smoke test on two configs and, from the logged `time`, estimate the duration of a full 40k-iteration run and the epochs it represents.

9 Evaluation

9.1 Metrics

`IoUMetric` computes, per class and averaged over classes:

Key	Definition
`IoU` / `mIoU`	TP / (TP + FP + FN)
`Fscore` / `mFscore`	2·P·R / (P + R) (Dice)
`Precision`, `Recall`	TP / (TP + FP), TP / (TP + FN)
`Acc` / `mAcc`	per-class pixel accuracy (= recall)
`aAcc`	overall pixel accuracy (dominated by background; not a model-selection metric)

`iou_metrics=['mIoU', 'mFscore']` requests both families. Pixels labeled 255 are excluded. For a two-class problem report the target-class IoU and F-score in addition to the means, as in the paper (the background IoU is always high).

9.2 Running the test

```
python tools/test.py projects/buildings/configs/upernet_r50.py work_dirs/buildings/upernet_r50 \
    --work-dir work_dirs/buildings/upernet_r50/test                    # folder -> best_mIoU_iter_*.pth
python tools/test.py <config> <ckpt> --test-split Generalizability       # other split
python tools/test.py <config> <ckpt> --out work_dirs/.../preds           # save index masks
python tools/test.py <config> <ckpt> --show-dir work_dirs/.../vis        # overlays
python tools/test.py <config> <ckpt> --tta                               # flips + scales
```

Test-time augmentation averages predictions over the `tta_pipeline` scales and flips; expect +0.3–1.0 mIoU at 6–12× the cost. Report with and without, and never compare a TTA result with a non-TTA one.

`mode='slide'` (window and stride in `test_cfg`) evaluates each tile in overlapping windows and averages logits; it matters when test tiles are larger than the training crop. `mode='whole'` runs the full tile at once.

9.3 Comparing runs

```
python tools/compare_runs.py work_dirs/buildings --out work_dirs/buildings/summary.md --csv summary.csv
```

run	best_iter	best_mIoU	best_mFscore	last_iter	last_mIoU	test_mIoU	test_mFscore
buildings/segformer_b2	36000	81.2	89.5	40000	80.9	80.1	88.7
buildings/deeplabv3plus_r50	30000	79.8	88.6	40000	79.5	78.6	87.9

Model selection uses `best_mIoU` on validation; the report uses the test split, evaluated once, with the selected checkpoint. Add parameters and FLOPs:

```
python tools/analysis_tools/get_flops.py projects/buildings/configs/segformer_b2.py --shape 512 512
```

9.4 Error analysis

Confusion matrix from saved predictions:

```
python tools/test.py <config> <ckpt> --out work_dirs/x/preds        # writes one index PNG per test image
python tools/analysis_tools/confusion_matrix.py <config> work_dirs/x/preds work_dirs/x/cm
```

The confusion-matrix tool pairs every file in the prediction folder with the test dataset in order, so the prediction folder must contain the test split only, produced by a single `test.py --out` run.

Visual review: `--show-dir` overlays; sort tiles by per-tile IoU (a short script over `--out` masks and the ground truth) and inspect the worst 20. The paper's Fig. 13 pattern applies to most projects: errors concentrate at shadows, mixed crowns/adjacent objects, low-contrast backgrounds and boundary pixels. Those categories decide the next action: more training examples of that condition, a different loss, or a post-processing rule.

9.5 Generalisability

A test set from the same regions as training measures interpolation. A held-out region (different date, terrain, sensor settings) measures what the model does on new surveys, which is the operational question. Keep one such region from the start (`make_tiles.py` per orthomosaic, hold one out), report both, and expect the gap to be larger for CNNs than for transformers/hybrids, as in the Acacia study.

9.6 Exercise 8

For a binary model with mIoU = 85 %, background IoU = 97 % and target IoU = 73 %: which number goes into the abstract, and why would `aAcc = 98 %` be misleading?

10 Prediction and export to GIS

10.1 Single images from Python

```
from mmseg.apis import init_model, inference_model, show_result_pyplot
model = init_model('projects/buildings/configs/segformer_b2.py',
                   'work_dirs/buildings/segformer_b2/best_mIoU_iter_36000.pth', device='cuda:0')
result = inference_model(model, '/data/buildings/img_dir/test/b12_000000000012.tif')
mask = result.pred_sem_seg.data[0].cpu().numpy()          # H x W class indices
show_result_pyplot(model, '/data/.../b12_000000000012.tif', result, out_file='pred.png', show=False)
```

`MMSegInferencer(model=config, weights=ckpt)` offers the same for folders (`inferencer(folder, out_dir='out', pred_out_dir='pred')`). Both use the config's `test_pipeline`, so they load with OpenCV: for multispectral data use the geospatial pipeline below or feed arrays through `LoadImageFromNDArray`.

10.2 Orthomosaics: the geospatial pipeline

`tools/geospatial_inference.py` (one raster) and `tools/batch_geospatial_inference.py` (a folder tree) implement sliding-window inference over gigapixel GeoTIFFs with any MMSegmentation config:

```
python tools/geospatial_inference.py \
  --config projects/buildings/configs/segformer_b2.py \
  --checkpoint work_dirs/buildings/segformer_b2 \
  --input /data/orthos/city_block_cog.tif --output /data/predictions/city_block_buildings.gpkg \
  --tile-size 512 --overlap 128 --thresh 0.5 --min-area 4.0 --connectivity 8 --save-prob
```

Steps: edge-aligned tiling with overlap; per-tile softmax of the target class; centre-crop (seam-free) or Hann blending; probability GeoTIFF in the input CRS; threshold; GDAL polygonisation; area filter; per-polygon `mean_prob`; GeoPackage or Shapefile. Parameters and their meaning are in `docs/05_inference.md`.

For multi-class models the script exports one class (`--class-id`); run it once per class of interest, or use `--save-prob` per class and combine the probability rasters in GIS (argmax). Bands other than 8-bit RGB: `--bands` and the model's normalisation are read from the config's `data_preprocessor`, so a 10-band model works unchanged provided the orthomosaic has the same band order and scaling as the training tiles.

10.3 Post-processing in GIS

Need	Operation
remove specks	`--min-area` (m²) at export, or *Select by Attributes* on `area`
keep confident objects	filter on `mean_prob` (e.g. ≥ 0.6)
separate touching crowns	watershed on the probability raster (SAGA/GRASS in QGIS), or an instance model
smooth building outlines	*Simplify Building* / *Regularize Building Footprint* (ArcGIS Pro)
road centrelines	thin the raster (*Thin* tool) and vectorise as lines
areas by class	*Tabulate Area* on the argmax raster

10.4 Throughput and memory

Inference memory is set by `--tile-size`, `--batch-size` and precision (`--precision fp16` default). A 24 GB GPU handles batch 8 of 1024² tiles for U-MV-small. I/O dominates on network drives: stage the input with `--scratch-dir /tmp/geospatial_work` and convert orthomosaics to COG first.

10.5 Exercise 9

Map one orthomosaic with `--blend center` and `--blend hann`, and compare the crown outlines along a tile boundary in QGIS (`--overlap 0` shows the seams).

11 Worked examples

Four projects, one pattern: copy the template, edit the dataset base, pick model configs, train, compare, map. Every config below is in `tutorial/examples/` and is loaded and built in the repository's test suite.

11.1 Acacia crowns from UAV RGB (binary, 2.5 cm, 1024² tiles)

The reference project. `tutorial/examples/acacia_rgb/configs/_base_/dataset.py` points the generic template at the hand-over dataset (`/data`, `test2` split, 1024² crops, batch 2) and five model configs share it, which allows the comparison of the paper to be repeated with one command:

```
bash tutorial/templates/project/scripts/run_experiments.sh tutorial/examples/acacia_rgb
python tools/compare_runs.py work_dirs/acacia_rgb
```

Design choices to notice: 1024² crops because a crown needs its shadow and neighbours; scale jitter 0.5–2.0; CE + 3·Dice; sliding-window testing with a 512 stride; the evaluation on a separate region (`Generalizability`).

11.2 Building footprints from aerial RGB (binary, 5–10 cm, 512² tiles)

`tutorial/examples/buildings_rgb/`. Differences: 512² crops (buildings are compact); CE + 2·Dice against the background majority; 8-connected polygonisation and `--min-area 4` at export; footprint regularisation in GIS afterwards. Suggested baselines: DeepLabV3+-R50 (sharp edges from the low-level skip) and SegFormer-B2. Typical pitfalls: shadows of tall buildings labeled as background create boundary errors — include them in the reference polygons consistently (roof outline, not shadow); flat roofs versus paved lots need context, so do not go below 512².

11.3 Roads (binary, thin connected structures, 768² tiles)

`tutorial/examples/roads_rgb/`. Differences: larger crops for continuity; vertical flips and ±30° rotation (no preferred orientation); CE + Lovasz (IoU surrogate, rewards connectedness); no minimum-area filter at export, and a thinning step to obtain centrelines. Evaluate the road IoU and, if connectivity matters, a topological measure computed in GIS (number of connected components versus the reference). SegFormer-B2 and UPerNet-R50 are the provided configs.

11.4 Land cover from 10-band Sentinel-2 (7 classes, uint16, 256² tiles)

`tutorial/examples/lulc_multispectral/`. This is the example to study for any non-RGB sensor. The dataset base uses `LoadRasterioImage` with reflectance scaling and clipping, drops `PhotoMetricDistortion`, sets per-band `mean`/`std` and `bgr_to_rgb=False`; the model configs set `in_channels=10`, disable the RGB visualisation hook and use CE + Dice. To keep ImageNet pretraining:

```
python tools/download_zoo_weights.py resnet50_v1c
python tools/adapt_first_conv.py work_dirs/pretrained/resnet50_v1c-2cccc1ad.pth work_dirs/pretrained/resnet50_v1
c_10band.pth \
    --key stem.0.weight --bands 10 --mode map --rgb-bands 2 1 0      # B4=R, B3=G, B2=B at stack positions 2,1,0
```

and set `backbone=dict(init_cfg=dict(type='Pretrained', checkpoint='work_dirs/pretrained/resnet50_v1c_10band.pth'))`. Compute the statistics with `tools/band_statistics.py` before training; the values in the example config are placeholders.

11.5 Reusing the Acacia model for a new tree species

Fine-tune rather than retrain: `load_from` the Acacia small checkpoint, new dataset base with the new species, learning rate 2e-5, 10k iterations (chapter 8.6). If the imagery is a different sensor or GSD, extend

`ratio_range` so that the training scale distribution covers the new GSD.

11.6 A checklist for any new task

1. Define classes and the ignore policy; write the class table into the dataset base.
2. Choose the tile size from object size × context at the working GSD.
3. Rasterise, tile with spatial-block splits, hold out one region.
4. `check_dataset.py`, `browse_dataset.py`.
5. Loader and normalisation for the sensor (chapter 6.6).
6. Two baselines from different families + the team model; smoke test; full runs.
7. `compare_runs.py`; error analysis; one round of targeted data or loss changes.
8. Test-split and held-out-region evaluation, reported once.
9. Map with the geospatial pipeline; post-process in GIS.
10. Record the experiment (chapter 12).

12 Team workflow

12.1 Project structure

```
U-MV-Acacia-tortilis-Crown-Mapping/          (the shared repository)
├── projects/<name>/                         one folder per project, committed
│   ├── configs/_base_/{dataset.py, schedule.py}
│   ├── configs/<model>.py ...
│   ├── scripts/run_experiments.sh
│   └── README.md                            data source, classes, GSD, decisions, results table
├── work_dirs/<name>/<model>/                outputs, never committed
└── /data/<name>/                            tiles, never committed
```

Start a project with `cp -r tutorial/templates/project projects/<name>` and commit the configs at every meaningful change. A config is a complete description of an experiment; a result without its config is not reproducible.

12.2 The experiment record

Keep one Markdown table per project (`projects/<name>/README.md`), one row per run, filled from `compare_runs.py`:

Run (work dir)	Config commit	Data version	Model	Crop / batch / iters	Val mIoU (best iter)	Test mIoU / mF	Notes
buildings/segformer_b2	a1b2c3d	tiles_v2 (2026-09-01)	SegFormer-B2	512 / 4 / 40k	81.2 (36k)	80.1 / 88.7	CE+2·Dice

Data version means the `tiles_index.csv` and the tiling command; store both with the dataset.

12.3 Sharing results

- Checkpoint of record: `best_mIoU_iter_*.pth`, copied with its resolved config and the run's `scalars.json` to the shared weights folder under `<project>/<model>/`. Publish with `tools/misc/publish_model.py` when a checkpoint leaves the team (removes optimiser state, adds a hash).
- Maps: GeoPackages with `mean_prob` so that thresholds can be revisited.
- Report figures: validation curves (`analyze_logs.py`), the comparison table, overlays of representative and failure cases.

12.4 Supervision milestones for a student project

Week	Deliverable	Evidence
1	environment verified; dataset tiled and checked	`verify_install.py` and `check_dataset.py` outputs
2	smoke test and first full run of one baseline	log, validation curve
3–4	three models compared under identical settings	`summary.md`, parameters/FLOPs
5	error analysis and one improvement round	confusion matrix, worst-tile gallery, before/after table
6	held-out region evaluation and a mapped orthomosaic	test table, GeoPackage in QGIS
7	written report with the experiment record	`projects/<name>/README.md`

12.5 Upgrades and getting help

Upgrading MMSegmentation or PyTorch means rebuilding the image, rerunning `tests/` and re-evaluating U-MV-small on `test2` (85.4 % mIoU). When asking for help, give the command, the config, the log tail and

`verify_install.py`.

A Cheat-sheets

A.1 Commands

```
# environment
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
python tools/verify_install.py [--variant small]
python tools/download_backbones.py            # MambaVision (HF)      python tools/download_zoo_weights.py   # R
esNet/MiT/Swin
# data
python tools/make_tiles.py --image O.tif --mask L.tif --out /data/P --tile 512 --split-blocks 4 --val-frac .15 -
-test-frac .15
python tools/check_dataset.py /data/P --splits train val test
python tools/band_statistics.py /data/P/img_dir/train --bands 1 2 3 4 --scale 1
python tools/analysis_tools/browse_dataset.py CFG --output-dir work_dirs/browse --not-show
# configs
python tools/misc/print_config.py CFG [--cfg-options k=v]
# training
python tools/train.py CFG --work-dir work_dirs/P/M [--amp] [--resume] [--cfg-options k=v ...]
bash tutorial/templates/project/scripts/run_experiments.sh projects/P [M1 M2 ...]
# evaluation
python tools/test.py CFG CKPT_OR_WORKDIR [--test-split S] [--tta] [--out DIR] [--show-dir DIR]
python tools/compare_runs.py work_dirs/P --out summary.md
python tools/analysis_tools/get_flops.py CFG --shape 512 512
python tools/analysis_tools/confusion_matrix.py CFG PRED_DIR OUT_DIR
python tools/analysis_tools/analyze_logs.py work_dirs/P/M/<ts>/vis_data/<ts>.json --keys mIoU loss
# prediction
python tools/geospatial_inference.py --config CFG --checkpoint CKPT --input ortho.tif --output out.gpkg [--save-
prob]
python tools/batch_geospatial_inference.py --config CFG --checkpoint CKPT --input-dir D --output-dir O --skip-ex
isting
python tools/inspect_checkpoint.py CKPT_OR_WORKDIR
```

A.2 Config keys most often changed

Key	Typical values
`train_dataloader.dataset.data_root`	`/data/<project>`
`train_dataloader.batch_size` / `num_workers`	2–8 / 4–16
`train_dataloader.dataset.pipeline[i].crop_size`	256–1024
`model.data_preprocessor.size`	= crop size
`model.data_preprocessor.mean/std/bgr_to_rgb`	ImageNet + True (RGB via OpenCV); statistics + False (rasterio)
`model.backbone.in_channels`	3, 4, 10
`model.decode_head.num_classes` (+ `auxiliary_head`)	classes incl. background
`model.decode_head.loss_decode`	CE; CE+Dice; CE+Lovasz
`model.test_cfg`	`dict(mode='slide', crop_size=..., stride=...)` or `dict(mode='whole')`
`optim_wrapper.optimizer.lr`	6e-5–1e-4 (AdamW), 1e-2 (SGD)
`train_cfg.max_iters` / `val_interval`	20k–160k / 5 %
`default_hooks.checkpoint.save_best`	`'mIoU'`
`load_from` / `resume`	checkpoint path / True
`randomness.seed`	0, 1, 2

A.3 Transform quick list

Loading: `LoadImageFromFile`, `LoadAnnotations`, `LoadSingleRSImageFromFile`, `LoadRasterioImage`, `LoadImageFromNDArray`. Geometry: `RandomResize`, `Resize`, `ResizeToMultiple`, `RandomCrop`, `RandomFlip`,

`RandomRotate`, `RandomRotFlip`, `RandomMosaic`, `RandomCutOut`. Radiometry (8-bit): `PhotoMetricDistortion`, `CLAHE`, `AdjustGamma`, `Rerange`, `RGB2Gray`, `Albu`. Formatting: `PackSegInputs`, `GenerateEdge`.

A.4 Registered names used in this tutorial

Backbones: `ResNetV1c`, `UNet`, `MixVisionTransformer`, `SwinTransformer`, `MambaVisionBackbone`. Heads: `FCNHead`, `PSPHead`, `DepthwiseSeparableASPPHead`, `UPerHead`, `SegformerHead`, `GenericUNetHead`. Losses: `CrossEntropyLoss`, `DiceLoss`, `LovaszLoss`, `FocalLoss`, `TverskyLoss`. Datasets: `BaseSegDataset`, `UAVAcaciaDataset`, `RepeatDataset`, `ConcatDataset`. Metrics: `IoUMetric`. Hooks: `CheckpointHook`, `LoggerHook`, `SegVisualizationHook`, `EarlyStoppingHook`. Schedulers: `LinearLR`, `PolyLR`, `CosineAnnealingLR`. Optimisers: `AdamW`, `SGD`.

B Troubleshooting

Symptom	Cause	Fix
`KeyError: 'X is not in the mmseg::model registry'`	custom module not imported, or typo	`custom_imports = dict(imports=['umv'])`; check spelling and scope prefix
`Duplicate key is not allowed among bases`	two `_base_` files define the same variable	keep helper variables in one base
`TypeError: string indices must be integers` / `'str' and 'int'` in a config	arithmetic on a `{{_base_.x}}` placeholder	compute in the base file, reference whole values
`RuntimeError: SyncBatchNorm ... single process`	`SyncBN` on one GPU	`norm_cfg = dict(type='BN', requires_grad=True)`
`size mismatch for decode_head.conv_seg.weight`	`num_classes` differs from the checkpoint	expected when transferring to new classes (warning); an error means the config is wrong
`IndexError: index 255 is out of bounds` in `cross_entropy`	`class_weight` with ignored pixels	remove `class_weight`; use Dice/Lovasz/Focal
`AssertionError: ... in_channels` / shape error at the first conv	`in_channels` ≠ bands produced by the loader	align `bands`, `in_channels`, `mean/std` lengths
all-background predictions, mIoU ≈ 50 % (binary)	masks 0/255 or RGB; wrong suffix; `reduce_zero_label=True` on a 0-based mask	`check_dataset.py`; set `reduce_zero_label=False`
loss NaN after a few hundred iterations	lr too high; no `clip_grad`; NaN/inf pixels in 16-bit input	lower lr, `clip_grad`, `nodata_fill`, `clip`
`CUDA out of memory`	batch/crop too large	`--amp`; smaller batch; smaller crop; `mode='slide'` at test
GPU utilisation low, `data_time` high	CPU workers or disk	`num_workers` up; SSD; COG; smaller tiles
`DataLoader worker ... Bus error`	container shared memory	run with `--ipc=host` (compose does)
WSL2 freezes when memory is full	sysmem fallback	NVIDIA Control Panel → Prefer No Sysmem Fallback
`OSError: cannot connect to huggingface.co`	offline	`tools/download_backbones.py` once; `HF_HUB_OFFLINE=1`
`open-mmlab://resnet50_v1c` download fails	offline	`tools/download_zoo_weights.py`; `pretrained='work_dirs/pretrained/resnet50_v1c-2cccc1ad.pth'`
overlays in `browse_dataset` look wrong for multispectral	3-band assumption	ignore or make RGB copies; `draw=False`
`PhotoMetricDistortion` error with float images	8-bit assumption	remove it for non-8-bit inputs
predictions shifted relative to the image in GIS	tiles written without georeferencing, or image/mask grid mismatch	use `make_tiles.py`; check `gdalinfo` of image and mask
test mIoU far below validation	leakage-free split reveals over-fitting, or different region	expected; report both; more diverse training data
`tools/test.py` finds no images	wrong `data_prefix` or split name	`--test-split`, `check_dataset.py`

C Glossary

Term	Definition
AMP	automatic mixed precision: 16-bit arithmetic where safe, 32-bit elsewhere; halves memory
auxiliary head	secondary classifier on an intermediate feature level used during training only
backbone / encoder	the network part that turns pixels into multi-scale features
BN / SyncBN / GN / LN	batch, synchronised batch, group and layer normalisation
checkpoint	file with model weights (and optimiser state) at one iteration
COG	cloud-optimised GeoTIFF: tiled, with overviews; fast windowed reads
config	Python file defining data, model, schedule and runtime of an experiment
data preprocessor	module that normalises, pads and stacks a batch on the GPU
decode head / decoder	the network part that turns features into per-pixel class scores
Dice loss	1 − 2·overlap/(sum of areas); region-based, imbalance-robust
FLOPs	floating-point operations of one forward pass; cost measure
GSD	ground sampling distance: ground size of one pixel
hook	callback executed at fixed points of the training loop (logging, checkpoints, ...)
ignore index	label (255) excluded from loss and metrics
IoU	intersection over union of predicted and reference pixels of a class
iteration	one optimiser step; MMSegmentation schedules count iterations
Lovasz loss	convex surrogate of the IoU, optimised directly
metainfo	class names and palette attached to a dataset
mIoU / mFscore	class-averaged IoU / F-score
pipeline	ordered list of transforms applied to each sample
registry	table mapping `type` strings to classes
sliding-window inference	predicting a large image in overlapping windows and merging
tile	fixed-size crop of a raster used as one sample
TTA	test-time augmentation: averaging predictions over flips and scales
work directory	output folder of a run

D Exercise solutions

1. Batch 2 × 100 000 iterations = 200 000 samples: 7.5 epochs over 26 615 tiles, 40.9 epochs over 4 893. On the smaller set the same iteration budget over-fits earlier; plan 20–40k iterations and rely on the best-validation checkpoint, or keep 100k with early stopping.

2. DeepLabV3+-R50-d8: backbone `ResNetV1c` with `out_indices=(0,1,2,3)` and dilations (1,1,2,4), so stage outputs are at strides 4, 8, 8, 8; `DepthwiseSeparableASPPHead` consumes `in_index=3` (2048 channels) plus the low-level `c1` features from stage 0 (256 channels); `FCNHead` auxiliary on `in_index=2` (1024 channels); losses CE 1.0 and CE 0.4. U-MV: backbone `MambaVisionBackbone`, head `GenericUNetHead` on all four levels, losses CE 1.0 + Dice 3.0, no auxiliary head.

3. (a) `--cfg-options optim_wrapper.optimizer.lr=5e-5`; (b) crop size, `model.data_preprocessor.size`, `RandomResize.scale`, `RandomCrop.crop_size` and `test_cfg` must change together — in the template only `crop_size` in `_base_/dataset.py` needs editing because the others derive from it; (c) `--cfg-options model.auxiliary_head=None`.

4. With 2 × 2 blocks one quarter of the raster is a contiguous test region (one landscape type, pessimistic and noisy); with 8 × 8 blocks the test tiles are interleaved with training tiles (optimistic). The 8 × 8 estimate is closer to interpolation within the surveyed area; performance on a new region is estimated only by holding out an entire orthomosaic.

5. `LoadRasterioImage(bands=[3, 2, 1, 4])` returns R, G, B, NIR; `bgr_to_rgb=False`; `mean=[123.675, 116.28, 103.53, m_nir]`, `std=[58.395, 57.12, 57.375, s_nir]` with the NIR statistics from `band_statistics.py --bands 4`; `backbone.in_channels=4` with an adapted first convolution (`adapt_first_conv.py --bands 4 --mode map --rgb-bands 0 1 2`).

6. MiT-B1: `embed_dims=64`, `num_layers=[2, 2, 2, 2]`, head `in_channels=[64, 128, 320, 512]`, checkpoint `mit_b1_20220624-02e5a6a1.pth`. At 512²: B1 13.7 M parameters / 15 GFLOPs, B2 24.7 M / 25 GFLOPs, DeepLabV3+-R50-d8 43.6 M / 176 GFLOPs (2 classes; `get_flops.py --shape 512 512`).

7. Time per iteration from the log (`time`) × 40 000; epochs = 40 000 × batch / tiles.

8. Report the target-class IoU (73 %) together with mIoU; `aAcc` is dominated by the background (97 % of pixels) and would be 98 % even for a model that misses a quarter of the targets.

9. With `--overlap 0` the centre-crop writer degenerates to independent tiles and outlines break at tile edges; with `--overlap 256` both blends give continuous outlines; Hann blending is marginally smoother on ambiguous crowns.